

Understanding Remote Access Fundamentals & Importance

Remote Access Basics: Remote access is a technology that allows authorized users to connect securely to a computer or network from a different location using the internet or a private network. It plays a vital role in modern IT infrastructure by enabling system administrators, employees, and organizations to access resources remotely without being physically present. Remote access improves business continuity, supports remote work, and simplifies system administration. Technologies such as Secure Shell (SSH), Remote Desktop Protocol (RDP), and Virtual Private Networks (VPNs) provide secure communication through encryption and authentication mechanisms. By ensuring confidentiality, integrity, and controlled access, remote access enables users to safely manage systems and access organizational resources from anywhere.

Introduction

Remote access allows authorized users to connect to systems, applications, or networks from outside the physical perimeter of an enterprise network. This lab introduces the foundational principles of remote access, evaluating why it is indispensable for modern organizations and the security implications involved.

Objectives

Aim: To develop a comprehensive understanding of remote access concepts, their significance in enterprise IT and cybersecurity, and to reinforce this knowledge through the practical implementation of a secure SSH connection between two Kali Linux systems for remote administration.

- Understand the fundamentals of Secure Shell (SSH) and its role in secure remote system administration.
- Configure and establish a secure SSH connection between two Kali Linux systems using the SSH protocol.
- Analyze the security features of SSH, including encrypted communication, authentication methods, and secure remote access.

Requirements

- A computer with two Kali Linux Virtual Machines (Client and Server) connected on the same network.
- Basic understanding of IP addressing, networking concepts, and user authentication.

Expected Output

- Successful establishment of a secure SSH connection between two Kali Linux virtual machines.
- Verification of remote command-line access by executing commands on the remote system through an encrypted SSH session.
- Improved understanding of remote access concepts, SSH communication, authentication mechanisms, and the practical workflow of secure remote system administration.

Step by Step Procedure(s)

Step-1: Prepare the Virtual Machine Environment:

Create two Kali Linux virtual machines, ensure they are connected to the same network, verify their IP addresses, and install the OpenSSH Server package on the server virtual machine.

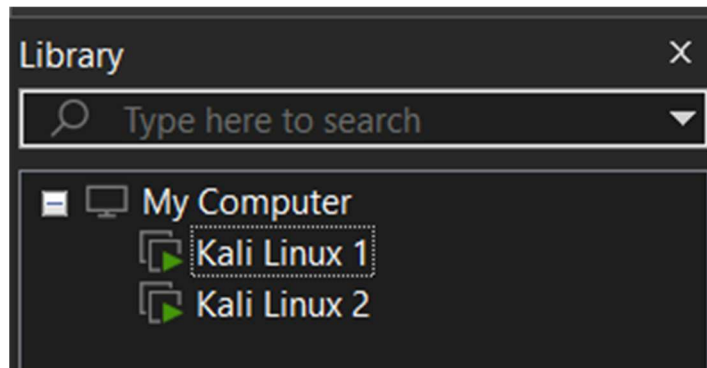


Fig: 1.1 Virtual Machines

Step-2: Install and Configure the OpenSSH Server:

Install the OpenSSH Server package on both virtual machines using `sudo apt install openssh-server`, enable and start the SSH service using `systemctl`, and verify that the SSH service is running successfully.

```
(kali@kali)~$ sudo apt install openssh-server -y
Upgrading:
  openssh-client  openssh-client-gssapi  openssh-server  openssh-sftp-server

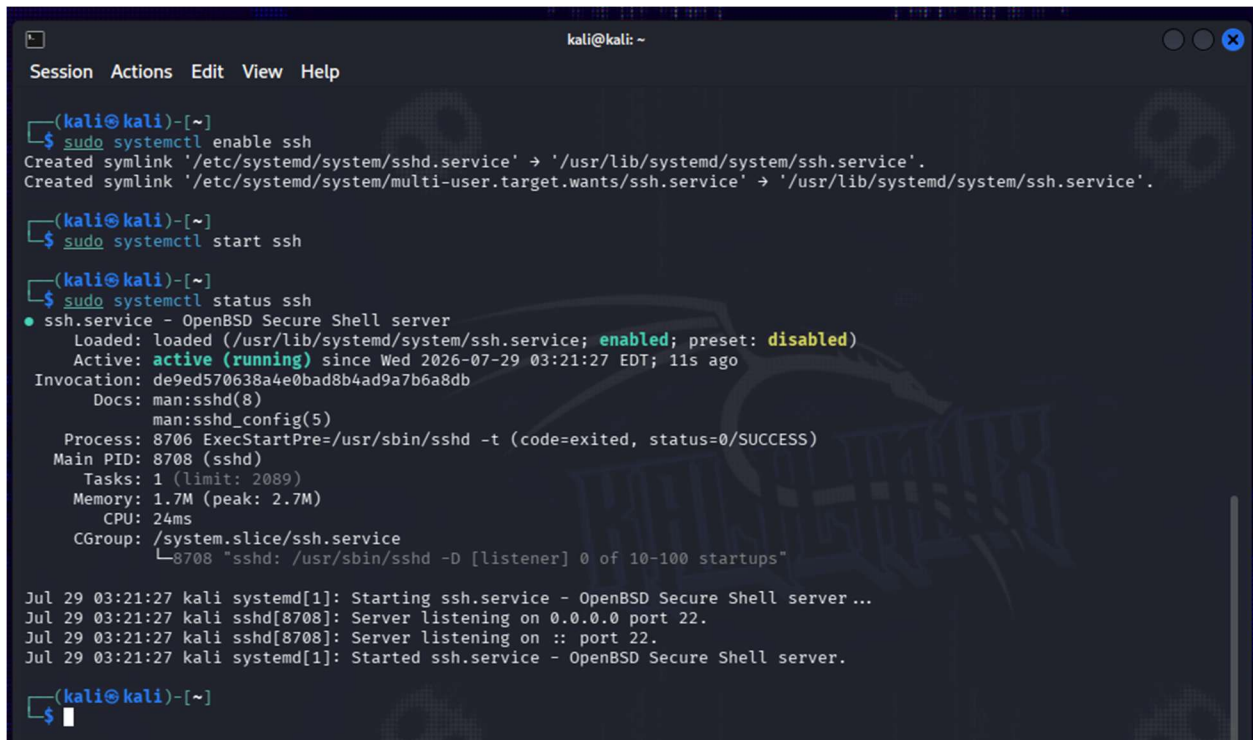
Summary:
  Upgrading: 4, Installing: 0, Removing: 0, Not Upgrading: 823
  Download size: 1,837 kB
  Space needed: 17.4 kB / 62.2 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 openssh-sftp-server amd64 1:10.3p1-5 [67.3 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 openssh-server amd64 1:10.3p1-5 [600 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 openssh-client amd64 1:10.3p1-5 [1,006 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 openssh-client-gssapi all 1:10.3p1-5 [164 kB]
Fetched 1,837 kB in 1s (1,712 kB/s)
Preconfiguring packages ...
(Reading database... 426184 files and directories currently installed.)
Preparing to unpack ./openssh-sftp-server_1%3a10.3p1-5_amd64.deb...
Unpacking openssh-sftp-server (1:10.3p1-5) over (1:10.3p1-4)...
Preparing to unpack ./openssh-server_1%3a10.3p1-5_amd64.deb...
Unpacking openssh-server (1:10.3p1-5) over (1:10.3p1-4)...
Preparing to unpack ./openssh-client_1%3a10.3p1-5_amd64.deb...
Unpacking openssh-client (1:10.3p1-5) over (1:10.3p1-4)...
Preparing to unpack ./openssh-client-gssapi_1%3a10.3p1-5_all.deb...
Unpacking openssh-client-gssapi (1:10.3p1-5) over (1:10.3p1-4)...
Setting up openssh-client (1:10.3p1-5)...
Setting up openssh-client-gssapi (1:10.3p1-5)...
Setting up openssh-sftp-server (1:10.3p1-5)...
```

Fig: 1.2 Installation of SSH

Step-3: Establish and Verify the SSH Connection:

From the client virtual machine, connect to the server using the ssh command and authenticate with the server user's credentials. After establishing the connection, execute the ip a (or ifconfig) command to verify the network interface details of the remote system and confirm successful remote access.

A terminal window titled 'kali@kali: ~' with a menu bar (Session, Actions, Edit, View, Help). The user runs 'sudo systemctl enable ssh', which creates symlinks. Then 'sudo systemctl start ssh' is run. Finally, 'sudo systemctl status ssh' is run, displaying detailed service status including 'Active: active (running)' and 'Main PID: 8708 (sshd)'. Log messages at the bottom show the service starting and listening on port 22.

```
(kali@kali)-[~]
└─$ sudo systemctl enable ssh
Created symlink '/etc/systemd/system/ssh.service' → '/usr/lib/systemd/system/ssh.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/ssh.service' → '/usr/lib/systemd/system/ssh.service'.

(kali@kali)-[~]
└─$ sudo systemctl start ssh

(kali@kali)-[~]
└─$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-07-29 03:21:27 EDT; 11s ago
     Invocation: de9ed570638a4e0bad8b4ad9a7b6a8db
       Docs: man:sshd(8)
            man:sshd_config(5)
    Process: 8706 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 8708 (sshd)
      Tasks: 1 (limit: 2089)
     Memory: 1.7M (peak: 2.7M)
        CPU: 24ms
    CGroup: /system.slice/ssh.service
            └─8708 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

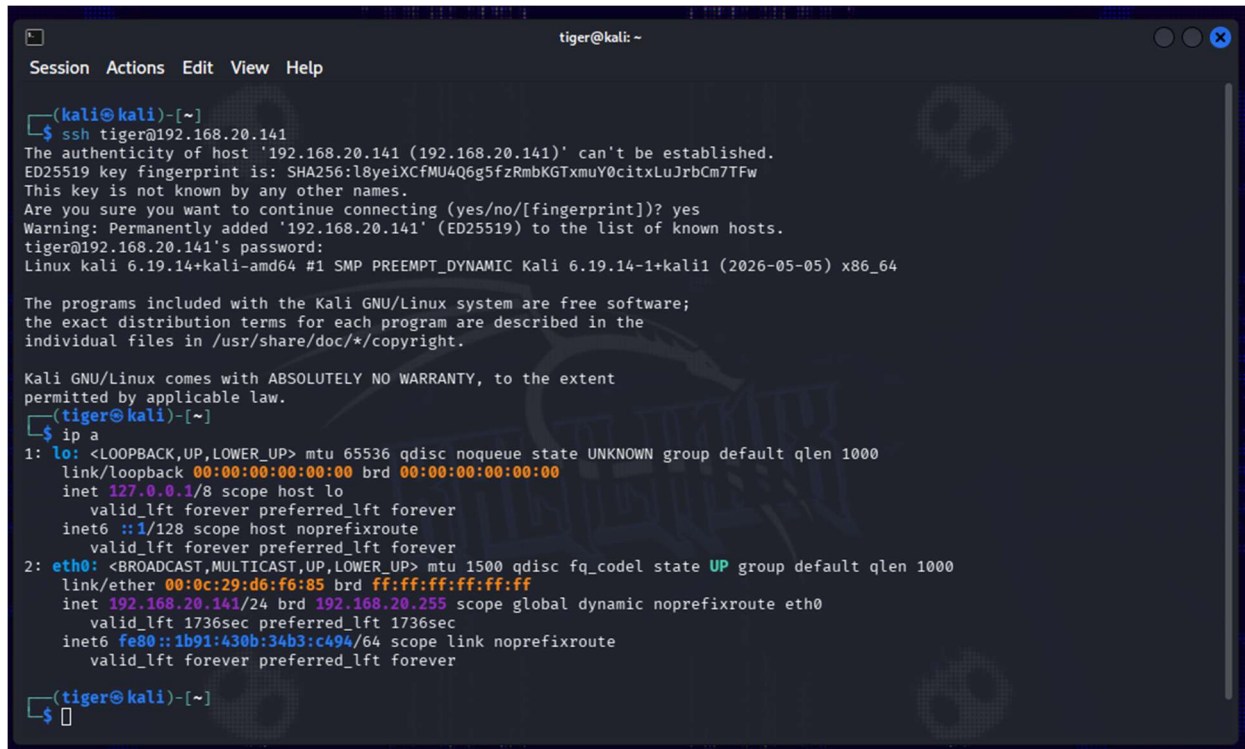
Jul 29 03:21:27 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server ...
Jul 29 03:21:27 kali sshd[8708]: Server listening on 0.0.0.0 port 22.
Jul 29 03:21:27 kali sshd[8708]: Server listening on :: port 22.
Jul 29 03:21:27 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.

(kali@kali)-[~]
└─$
```

Fig 1.3 Enabling SSH

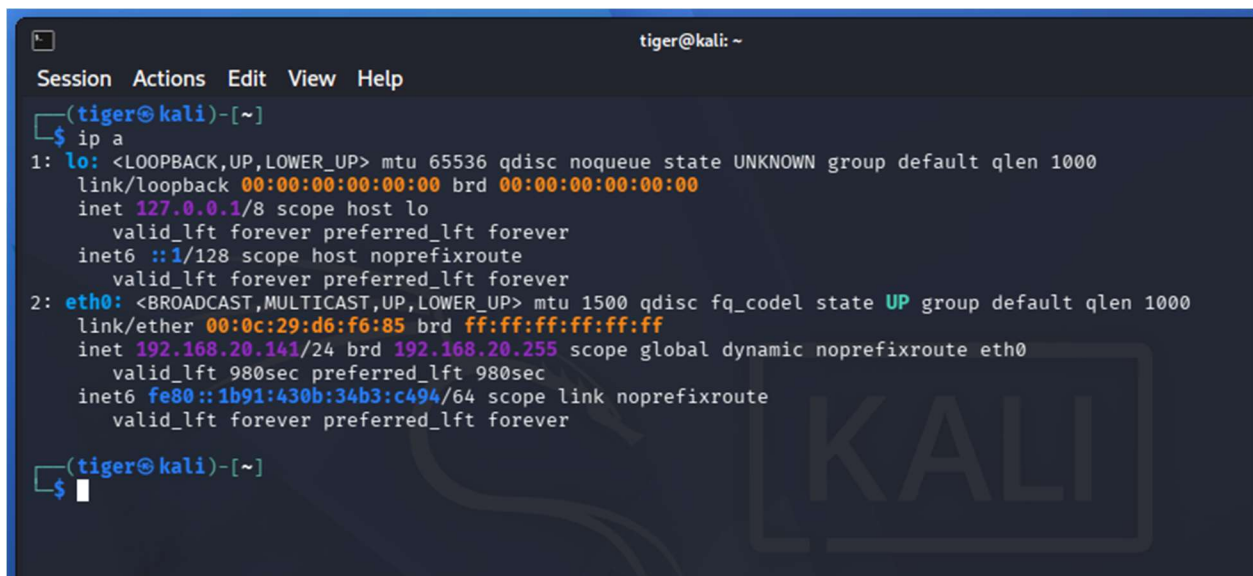
Step-4: Analyze Remote Access and SSH Security:

Observe how SSH establishes an encrypted communication channel between the client and server, analyze the authentication process, and understand how SSH provides secure remote administration in enterprise IT and cybersecurity environments.



```
tiger@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ ssh tiger@192.168.20.141  
The authenticity of host '192.168.20.141 (192.168.20.141)' can't be established.  
ED25519 key fingerprint is: SHA256:l8yeiXCfMU4Q6g5fzRmbKGTxmuY0citxLuJrbCm7TFw  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '192.168.20.141' (ED25519) to the list of known hosts.  
tiger@192.168.20.141's password:  
Linux kali 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1+kali1 (2026-05-05) x86_64  
  
The programs included with the Kali GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
(tiger@kali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 00:0c:29:d6:f6:85 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.20.141/24 brd 192.168.20.255 scope global dynamic noprefixroute eth0  
        valid_lft 1736sec preferred_lft 1736sec  
    inet6 fe80::1b91:430b:34b3:c494/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
(tiger@kali)-[~]  
$
```

Fig 1.4 Logging into Client



```
tiger@kali: ~  
Session Actions Edit View Help  
(tiger@kali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 00:0c:29:d6:f6:85 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.20.141/24 brd 192.168.20.255 scope global dynamic noprefixroute eth0  
        valid_lft 980sec preferred_lft 980sec  
    inet6 fe80::1b91:430b:34b3:c494/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
(tiger@kali)-[~]  
$
```

Fig 1.5 Verifying ip of client

Learning Outcomes

- Understand the fundamentals of remote access and the role of Secure Shell (SSH) in secure remote system administration.
- Successfully configure and establish a secure SSH connection between two Kali Linux virtual machines for remote command-line access.
- Analyze the importance of encrypted communication and authentication mechanisms in ensuring secure remote connectivity within enterprise IT and cybersecurity environments.

Notes/Observations

The OpenSSH Server was successfully installed and configured on the server Kali Linux virtual machine.

Both Kali Linux virtual machines were connected to the same network and could communicate using their IP addresses.

A secure SSH connection was successfully established between the client and server using the server's IP address and valid user credentials.

After authentication, the client gained secure remote command-line access to the server and executed the `ip a` command to verify the remote system's network configuration.

SSH communication was encrypted, ensuring secure transmission of data between the client and the server.

The experiment demonstrated the practical implementation of remote access and highlighted the importance of secure authentication and encrypted communication in enterprise IT and cybersecurity environments.